

# 重大 AI 更新提醒

09-19 | llama.cpp 修复 CUDA 排序损坏，避免 top\_k 采样出现越界崩溃

## 本期导读

本期聚焦 llama.cpp 的一次关键 CUDA 修复：CUB 的 argsort 在就地键缓冲区上会输出损坏的排列，导致 top\_k 采样偶发越界。该问题影响 Maxwell 架构、CUDA 12.5 与 CCCL 2.x 环境下的推理稳定性，修复后六个调用点全部改用独立输出缓冲区。

## 已检查来源

GitHub Release

## 1. llama.cpp b11047 修复 CUDA CUB argsort 就地键缓冲区导致排序损坏

来源 GitHub Release | 类型 GitHub Release | 主题 智能体与开发者平台 | 时间 09-19 02:24 | 分数 7

### 是什么

这是 ggml-org/llama.cpp 的 GitHub Release b11047，核心变更是修复 CUDA 后端中 argsort\_f32\_i32\_cuda\_cub 的排序损坏问题。该函数此前调用 CUB 的一次性 DeviceRadixSort::SortPairs API 时，把输入键缓冲区和输出键缓冲区传成了同一个 temp\_keys，也就是 d\_keys\_in == d\_keys\_out。CUB 内部采用双缓冲乒乓机制，要求输入和输出键缓冲区必须是不同的内存区域；当两者别名重叠时，排序过程会在某一轮中部分覆盖自己的输入，最终产生损坏的排列结果。修复方式是为全部六个调用点（普通与分段、升序与降序、size-query 与 execute）都使用独立的 keys-out 缓冲区。

### 通俗解释

可以把 CUB 的排序想象成洗牌：它需要一张 原牌堆 和一张 新牌堆，一边看原牌一边把牌按顺序放进新牌堆。如果原牌堆和新牌堆是同一张桌子，洗到一半就会把还没看的牌覆盖掉，最后得到的顺序是错的。llama.cpp 之前就犯了这个错，把输入和输出指向了同一块显存。结果是在 Maxwell 显卡、CUDA 12.5、CCCL 2.x 环境下，对 24.8 万列的词汇表做 top\_k 采样时，偶尔会返回垃圾索引，下游 get\_rows 再按这些索引取数据就会越界，表现为间歇性崩溃或乱码输出。修复后输入和输出各用一块独立显存，排序结果就正确了。

### 主要作用

解决 CUDA 推理中 top\_k 采样偶发返回垃圾索引并触发下游越界访问的问题，提升 llama.cpp 在 Maxwell 架构、CUDA 12.5 与 CCCL 2.x 组合下的数值正确性和运行稳定性。对于依赖 GPU 做本地大模型推理、且词汇表规模很大的用户，这个修复直接关系到采样结果是否可信、进程是否会崩溃。

### 核心功能

第一，修复覆盖全部六个 argsort 调用点，包括普通与分段排序、升序与降序、size-query 与 execute，避免只修一半留下隐患。第二，明确区分输入键缓冲区与输出键缓冲区，符合 CUB 双缓冲乒乓机制对内存不重叠的要求。第三，针对大词汇表场景，例如 248k 列的 top\_k 操作，消除间歇性垃圾索引。第四，阻止损坏排列向下游 get\_rows 传播，避免越界 gather 引发的崩溃。

### 对比判断

候选摘要中明确给出的对比是：修复前在 Maxwell/CUDA 12.5/CCCL 2.x 环境下，对 248k 列词汇表做 backend top\_k 会间歇性产生垃圾索引并触发下游 get\_rows 越界；修复后使用独立 keys-out 缓冲区，排序不再自我覆盖。除此之外暂无可信公开对比数据。

### 适合谁

适合在 Maxwell 架构 GPU 上运行 llama.cpp 的开发者、使用 CUDA 12.5 与 CCCL 2.x 工具链的推理部署团队、以及维护自定义 CUDA 算子或调用 CUB DeviceRadixSort 的底层工程师。对做本地大模型推理、关注采样正确性与稳定性的研究者同样值得关注。

直达链接 <https://github.com/ggml-org/llama.cpp/releases/tag/b11047>

本期仅收录 1 条候选，内容基于 llama.cpp 官方 release 说明整理，未引入外部数据。